What it took: methods, failures, and costs of an AI-driven replication in computational number theory

Author: Derek Marshall (ORCID 0009-0001-8101-3908).

Companion to: *A pre-registered replication of the Sawin–Sutherland height-ordered murmuration*, DOI 10.5281/zenodo.21443321. The replication note states the mathematics. This document states how the work was done, what it cost, where it failed, and what caught the failures. Every claim here cites a commit, an erratum row, a ledger entry, or a measured total in the repository's methods dossiers.

---

1. Why

I wanted to see how well I could drive a frontier language model to write C++ that studies prime numbers. A paper cited during early research seemed interesting and I wanted to reproduce it. The subject felt reminiscent of results from string theory, and that instinct turned out to be literal: murmurations were discovered in 2022 by He, Lee, Oliver, and Pozdnyakov, applying machine-learning techniques from mathematical physics to the large public databases of elliptic curves. The paper I set out to replicate is the 2025 Sawin–Sutherland conjecture (arXiv:2504.12295) that grew out of that discovery: an explicit candidate density for the murmuration of elliptic curves ordered by naive height.

There was no point beyond the question itself: is it possible to reach a correct result in an extremely complex domain, without being a direct expert in the details, by directing language models under a discipline strict enough to catch their errors and mine. The answer is yes, conditionally. The conditions are this paper.

The replication succeeded (at least until I am proven wrong by an expert in the field). The three shape invariants of the empirical murmuration agree with the corrected conjectured density within the pre-registered tolerance, at the bottom of the height window the authors studied. Along the way the project produced a wrong headline result, sustained it for eight days through every internal check I had built, and then caught it. That sequence, not the mathematics, is the useful output.

2. The architecture

The build ran in four stages, across three vendors' models, with one human in the loop.

Stage 1: landscape research. Google Gemini Deep Research produced the initial survey of the murmuration literature and the candidate papers. This stage chose the target.

Stage 2: orchestration. A Claude (Fable) session in the web interface acted as the standing orchestrator. It held the project state, triaged findings, drafted the prompts for the build sessions, and was one of the referees. I relayed its output to the builders and the builders' output back.

Stage 3: build. Claude Code sessions running Opus at high effort did the work: the C++, the runs, the documents, the ledgers, the fixes. This is where nearly all tokens were spent.

Stage 4: secondary review. Gemini reviewed selected outputs. Its observations caught real defects (three errata rows credit external model review). Its proposed wordings consistently failed the project's own style

gates (which could have been better organized and documented at the outset so that all agents could use the same playbook; section 7, rule 12), and it once cited a repository file that did not exist. It was useful as a defect detector. Cross-vendor review earns its place in the table below on the detector side alone.

A session-level analysis of my own prompts (methods dossier, session-shapes section) classifies my keystrokes: roughly forty percent relaying prepared prompts, the rest split among approvals, corrections, questions, and decisions. By keystroke count I was a router. The decisions were another matter. Every fork in the project terminated at me: which specs to pin, which deviations to ratify, which drafted prompt to send and which to sit on, when a paragraph read wrong, when to stop a session, when to merge, when to mint. The prompts themselves were drafted by the orchestrator model. I selected, edited, and vetoed. The model drafted everything in this setup, including its own instructions, while the human kept control of every gate. The retrospective corrections to my own process are folded into section 7 as rules 9 through 12.

The decision ledger, kept separately, sorts my interventions into four kinds: scientific gates (spec approvals, deviation ratifications, one reversal of my own prior ruling), editorial judgments (title, disclosure wording, three occasions of "I do not like this paragraph," each of which produced a standing rule), process calls (fresh-session boundaries, commissioning the external reviews, stopping compute), and the irreversibles.

3. The machinery

The repository ran under working rules committed before the science started. Each rule below is listed with what it caught, cited to the errata ledger and the mechanisms dossier, and with its observable cost.

Pre-registration, commit before run. Every decision rule, tolerance, and threshold was committed to version control before the data it judges existed, citable afterward as ordered commit pairs. A cold review of the repository at v1.0.0 independently re-derived every claimed ordering from the git graph and confirmed them. The one ordering the graph cannot prove is the one the notes themselves decline to claim. Cost: one commit per rule, plus the discipline of writing rules before wanting them.

Twin implementations. Every function the result depends on had two independent implementations (a different algorithm, never a refactor) or an external oracle. The Frobenius trace ended up with three algorithms agreeing exactly over hundreds of millions of values. Twins caught implementation errors at a median latency of zero days. Cost: roughly double the implementation effort on the covered paths.

Oracles as referees only. PARI, LMFDB, and published values check the computation and never feed it; an absent oracle skips visibly instead of passing silently. This rule decided a fork late in the project: a proposed table of oracle-generated constants inside the density evaluator was rejected because it would have put referee provenance in the computational core. Cost: occasional extra implementation where a lookup would have been easier.

No constants from memory. Models state numbers fluently whether or not the numbers are real. Every constant in the repository is generated, transcribed from a cited source, or computed. Cost: negligible. Benefit: several, including one case where a from-memory curve count would have been wrong. There was one case where the Opus agent hilariously recited pi from memory and produced 3.14159265358979311600: the decimal expansion of the nearest double, printed past its own precision. The number looked funny enough that I stopped the session, which produced this rule, the constant-generation remedy (erratum #25, commit fecb9d9), and a substantial amount of refactoring.

Claims ledgers. Every sentence in the published note carries a marker tied to a ledger row citing the artifact that supports it (49 markers at release). The ledger caught sourcing drift repeatedly. It did not catch a mis-rounded table cell; recomputation did. Ledgers verify sourcing; verifying arithmetic takes recomputation.

Deviations as deliverables. Any departure from a committed plan gets its own commit stating the change and the reason, before proceeding. The ledger holds 31 numbered errata at v1.0.0. None were smoothed over. Several are the most informative objects in the repository.

Guard tests. Tests written against anticipated failure modes, before the failures exist. Most fired immediately or never. One (a copy-drift check between a canonical file and its byte-copy) sat silent for 66 commits

and then fired at the final release gate, catching a fix that had been applied to one copy of a formula and not the other. It was the last bug found before the tag.

A style checklist for published prose. The note targets a register in which a skeptical author of the source paper finds nothing to correct. The checklist began as six rules against catalogued tells of machine-generated text and grew to eight during the project (section 7). Fresh model prose regressed to the tells every time it was written without the checklist applied afterward. The working conclusion: the style pass is a post-condition of writing, not a phase of the project.

4. What the machinery could not catch

The headline result of the note was wrong for eight days, and every mechanism in section 3 passed it.

The note compares an empirical curve against a conjectured density. Our reading of the density's defining equation carried a transcription error: two products of local factors were transposed, and one exponent was mis-set. The error entered a pinning note early in the project. The C++ evaluator was built from that note. An independent PARI/GP oracle was built from the same note. The two implementations agreed to high precision, at every check, on the wrong function.

Every internal mechanism then did its job correctly on the corrupted object. The transcription check verified the note's formula character by character against the pinning note, confirming fidelity to an internal copy rather than to the source. The twin check verified that two implementations agreed, which only confirmed that both had been built from the same wrong reading. A truncation study, run in response to an external referee's concern, verified that the wrong density's trough was stable across a 24-fold range of cutoffs: an accurate convergence study of a formula the source paper never wrote. The pre-registered decision rule gated the empirical trough against the wrong target and returned its verdict honestly. The result read as a persistent open deviation from the conjecture. It was a bug.

The detection latencies in the errata ledger are bimodal. Errors caught by internal machinery (implementation slips, propagation misses, prose drift) have a median latency of zero days. The two errors in the shared-reading class, the transposed products and a related expired approximation bound, took eight to nine days and over a hundred commits each to surface. The internal machinery is nearly free and nearly instant. The errors it structurally cannot catch are the expensive ones, because every internal check verifies consistency with a shared source, and when that shared source is corrupted, consistency guarantees nothing.

The same class produced a second, subtler failure. A weight cutoff in the local factors had been justified as negligible, with the bound checked and true under the corrupted formula. Correcting the formula changed which terms mattered, and the justification silently expired. An approximation's validity certificate is conditional on the formula it was certified against.

5. What caught it

Everything that caught the expensive errors came from outside the shared context.

An adversarial referee pass. A fresh model session was given only the rendered note and told to review it as a skeptical author of the source paper. Its report identified the framing problem (comparing a finite-height curve against an asymptotic density and calling the gap a deviation) and demanded the truncation study. The study was methodologically sound and still could not find the bug, because the bug was upstream of it. But the referee round put the density's provenance under pressure for the first time.

A published figure. The source paper's own density plot, digitized and read numerically, put the density trough far from where our implementations put it. Two internally consistent implementations against one external artifact: the artifact was right. This was the first check in the project whose provenance did not pass through our own reading of the equation.

A blind re-transcription. A separate session that had never seen our code, our notes, or our reading of the formula transcribed the equation directly from the paper. The diff against our version identified both

errors in one pass. The corrected implementations, rebuilt from the paper rather than patched, landed on the published figure's trough exactly.

A cold clone. A fresh model session with no project context cloned the public repository and followed the note's reproduction instructions literally. It found every place the instructions assumed knowledge a stranger lacks, and its clean-clone build was the gate at which the 66-commit guard test finally fired.

The census over all 31 errata rows is one-sided: internal mechanisms caught the cheap errors, external mechanisms caught all of the expensive ones. The correction to the project's rules was structural, not incremental: a transcription check terminates at the source artifact and never at an internal quotation of it, and external context-free review is part of the machinery, not quality assurance applied to it.

One number belongs here. The three external review conversations together cost an estimated few tens of dollars, the cheapest category in the entire budget, and they were what corrected the headline result.

6. What it cost

All figures are transferable equivalents, not bills. The build ran on Claude Team subscription seats and my own hardware; the tables price the measured usage at published API rates and list AWS on-demand rates for the compute, so that someone estimating a similar project has numbers to scale from. Sources and methods are in the repository's usage and costs dossiers.

Tokens (build, measured). The Claude Code sessions made 3,857 API round-trips over the project. Measured usage and API-equivalent pricing (Opus, July 2026 rates):

Category	Tokens	Rate	Cost
Input (non-cached)	197,683	\$5/M	~\$1
Output	7,466,688	\$25/M	~\$187
Cache write	32,421,769	\$6.25/M	~\$203
Cache read	1,793,674,751	\$0.50/M	~\$897
Build total			~\$1,288

Cache reads dominate. That column is the model re-reading its accumulated context on every call, roughly 465,000 tokens per round-trip, priced at a tenth of fresh input. The non-cached input row is the human-side steering signal entering the build: under 200,000 tokens against 7.5 million of output. That steering was about 2.6 percent of the volume of the work it directed.

Tokens (orchestration and review, estimated). The web sessions (orchestration, referee rounds, cold-clone review, blind transcription) total an estimated 1.29 million tokens by character count, with a stated ±30 percent band and no input/output split (the export path stripped speaker roles). Priced at the bounding cases, the web layer adds roughly \$15 to \$65. The orchestration layer ran about 17 percent of the build's input-plus-output volume: a measured answer to how much steering the architecture in section 2 requires.

Compute (recorded). All jobs, priced at us-east-1 on-demand equivalents (rates retrieved 2026-07-19; the work actually ran on owned hardware):

Class	Hours	Equivalent	Cost
48-core box jobs	17.2	c7a.12xlarge, \$2.46/h	~\$42
Laptop jobs	25.3	c7a.4xlarge, \$0.82/h	~\$21
CI	14.7	public repo, free	\$0
Compute total			~\$63

Grand total: roughly \$1,350, with a stated ceiling near \$1,400. For scale: the project's entire scientific compute is on the order of 0.13 CPU-years. Sawin and Sutherland's computation at the top of the

height window, the one this project replicates the bottom of, is reported at 140 CPU-years. Against that scale the replication cost a rounding error, and the audit that corrected it cost a rounding error against the replication.

7. The rules I would start with next time

The project began with a rule set and ended with a larger one. The delta is the learnings, generated mechanically by diffing the committed rules at kickoff against the rules at v1.0.0.

1. A transcription check terminates at the source artifact (the paper itself), never at an internal quotation of it. Adopted after erratum #28.
2. Internal consistency machinery must be paired with external, context-free review as standing practice: an adversarial referee pass on the rendered artifact, and a cold-clone reproduction, both by sessions with no project context, before any release. The blind spots of a collaboration are shared by everyone inside it, including the human.
3. Any edit to source re-runs its affected test suites before the change is reported done. Adopted after a fix shipped against one copy of a formula and a guard test caught the other copy 66 commits later.
4. When a formula is corrected, every downstream approximation whose bound was certified under the old reading is re-audited. Validity certificates expire silently.
5. Oracle-provenance values never enter the computational core, even as pinned tables. That convenience is the same shared-source failure mode in another form.
6. A style pass over published prose is a post-condition of writing it, applied to exactly the sentences just written. Fresh model prose regresses to catalogued tells without it. Two rules joined the checklist mid-project: no minted aphorisms, and no dramatic metaphor.
7. Register-critical prose is never drafted at the tail of a long session. The builder sessions learned to refuse this on their own; the refusals were correct every time.
8. Numbers come from committed artifacts or from recomputation, never from memory and never from prose. Ledgers verify sourcing; verifying arithmetic takes recomputation.
9. Select the production algorithm before generating bulk data. The point-counting provider ran at roughly 145 times less CPU than the character sum it replaced, and the early caches were generated under the slow provider; the cost of that ordering is visible in the compute table as laptop hours.
10. Keep the continuous-integration loop fast from the first commit. Every gate in the project waits on it; the recorded CI time (14.7 hours) is a floor on that waiting.
11. Commit the code style guide at kickoff. Once results freeze, the code is evidence: a late restyle would have broken the validation chain, so the right to fix style is lost exactly when the urge to fix it arrives.
12. The prose style playbook is a kickoff artifact: versioned, numbered, and handed to every agent in the pipeline, internal and external. This project's house style existed as prose until late and was formalized as numbered rules only near release, which is part of why external review kept failing it.

The next iteration of the project is to restructure the repository as an easily reusable and reliable library, under the rules above from the first commit.

What this project demonstrates: one non-expert, directing frontier models under the discipline above, replicated a contemporary conjecture's numerical evidence in a domain he could not have entered alone, caught a headline-inverting error before publication, and shipped the corrected result with its full audit trail for about the price of a conference registration. What it does not demonstrate: that this generalizes. It is one project, one domain, one person, $n = 1$, and the discipline was most of the work. The note claims replication and constraint, not proof. This companion sets out a method and its receipts, and claims nothing more.